

Group Members: Eric Nelson and Pallavi Desai

1. What are we working on?

We built a discrete-event simulation using SimPy to model a distributed key-value store with hundreds of nodes under different churn patterns. Our first step is to compare Dotted Version Vectors (DVV) against standard vector clocks since DVV separates individual events, dots, from the aggregate causal context, the summary vector, which helps avoid metadata growth under concurrent updates. We will then implement our lease-based DVV, which extends DVV by attaching an expiration time to each summary-vector entry and garbage-collecting entries for nodes that fail to send heartbeats within the lease window. This yields time-bounded causal tracking so that once a lease expires, we stop tracking dependencies for that node, intentionally trading full causal coverage for smaller metadata under high churn. Our final report will quantify this tradeoff between causal guarantees and metadata bloat due to churn.

2. What is the central idea of our project?

The main idea of our project is to study how different causal metadata schemes behave under high membership churn, and to design a time-bounded variant that deliberately trades some causal precision for much smaller metadata. These patterns are particularly common in serverless architectures, where platforms rapidly spin up and tear down stateless functions backed by metadata caches or key-value stores. In such settings, a causally consistent metadata store that can scale to many short-lived replicas without incurring prohibitive causal metadata overhead is especially valuable. Although our evaluation is simulation-based, our results are intended to inform the design of causally consistent metadata stores and caches for highly elastic, serverless platforms, where functions experience rapid scaling and frequent placement changes.

3. What is new (novel)?

We have completed a basic event simulation framework to benchmark and compare different casual-clock mechanisms in a replicated key-value store under different models of changing membership. The current model supports stable, low-churn, sustained churn, and bursty churn environments generated randomly by sampling an exponential distribution from a set seed. As the simulation runs we collect message metadata and clock state and save them into CSVs for a separate post analysis script to analyze and convert into comparison charts automatically (see early results below). To keep the results deterministic all random samples are generated from a seed and each experiment is defined as a YAML configuration. Next we hope to add a casual verification script as we begin to further explore our lease-DVV solution.

We are using the simulator to compare standard vector clocks, Dotted Version Vectors (DVV), and a lease-based DVV extension on systems-level metrics (in progress). Our experiment pipeline

already tracks metadata per message, metadata growth under churn, latency statistics, and dependency queue length from buffered causal delivery. This gives us a common framework for measuring both overhead and correctness tradeoffs across clock designs.

We have completed the vector clock and DVV implementation as there were no viable DVV implementations in Python for this experiment. We have initial results for our DVV lease implementation with an aggressive pruning strategy, although we are actively developing ways to create better casual cuts in the clock history and measure the impact of these decisions to determine the causal horizon of our lease-DVV.

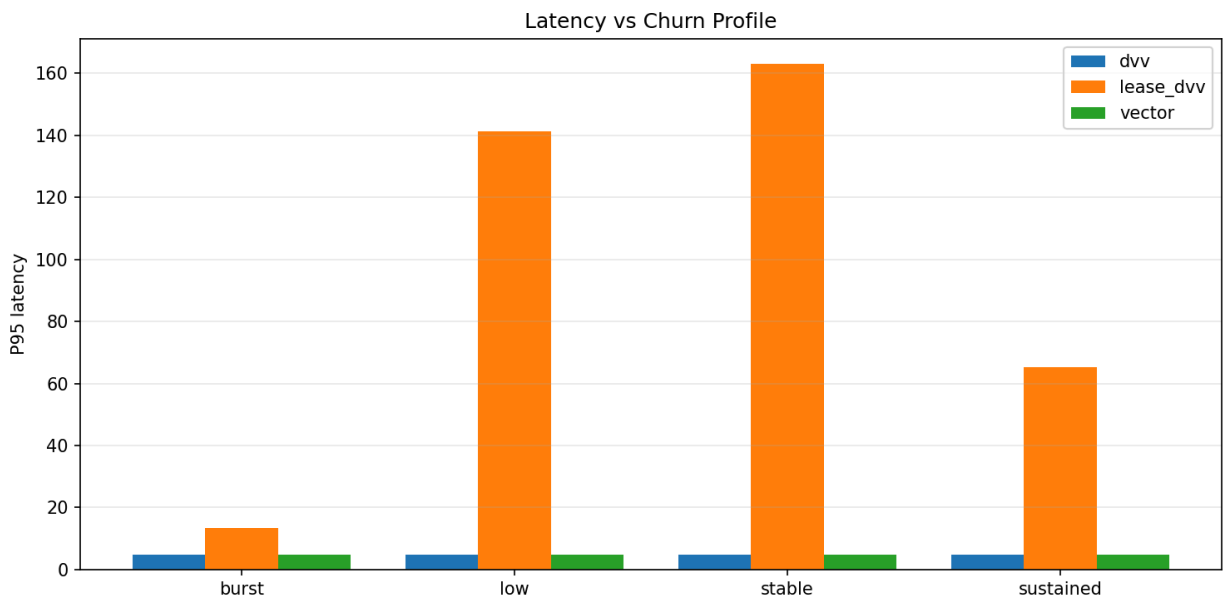
4. What have we done to validate it so far?

The following plots and tables have been generated by our current simulation framework showing early results for our vector clock, DVV and lease-DVV implementation:

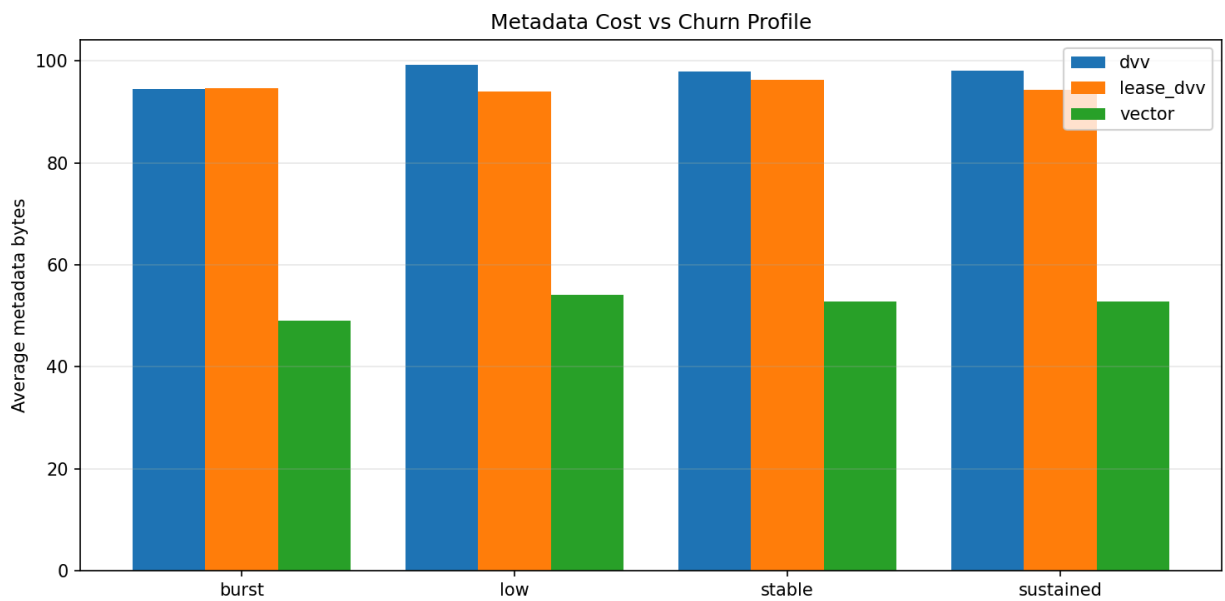
Visualizing simulation churn profiles (burst, low, stable, sustained)-churn all configurable:



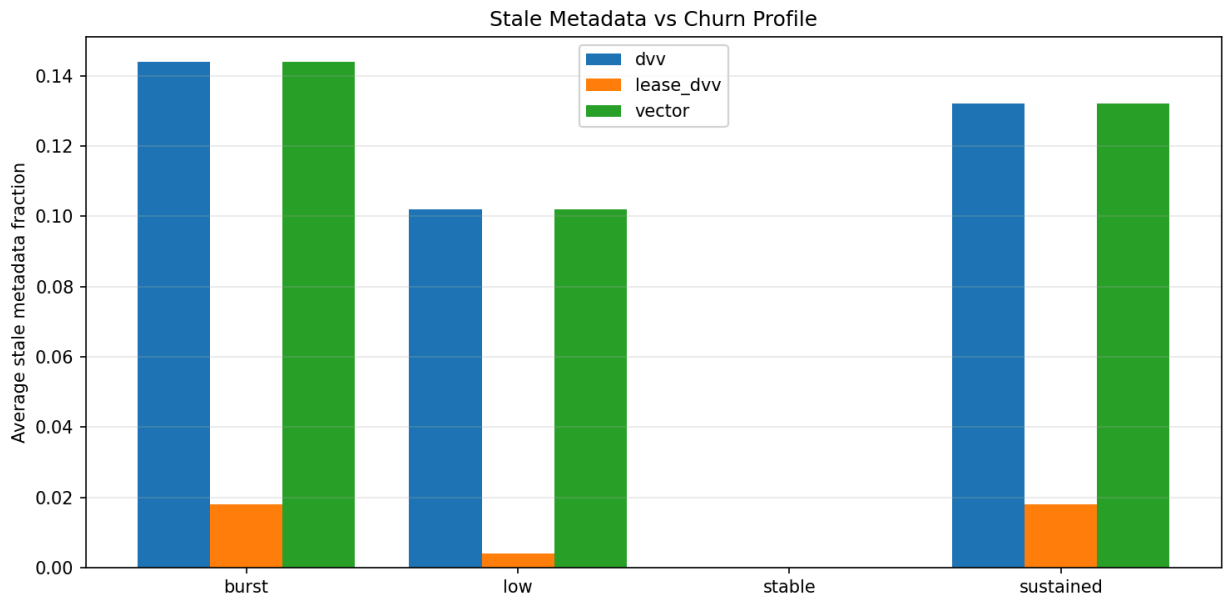
Since our simulation is event based we still need to implement an async garbage collection / lease mechanism as our current lease-DVV implementation checks and prunes old messages **during** processing inbound messages leading to a very high 95th percentile latency (ms) as seen here:



Visualizing the additional overhead of the dot + summary and dict based metadata of DVV vs vanilla vector clocks. Our hope is our optimized lease-DVV can close this gap:



Our most promising result thus far comes from comparing “stale” metadata across clock implementation resulting in our theoretical best pruning case for lease-DVV, showing that it aggressively avoids storing obsolete membership information. Our next steps will be to refine this pruning strategy and instead of simply marking meta-data to avoid transmitting stale data so we can see better overall latency and throughput statistics.



Lease-DVV stable clock state growth over time (sustained churn)

